Structural Field Theory (SFT): A Verification-First Protocol

A verification-ready protocol for testing a single-field structural-medium model (v0.1 draft)

Authors

Queral Rallo, Francisco - https://orcid.org/0009-0006-5326-0537

1: Structural Field Theory (SFT): A Verification-First Protocol

2: Appendix A — Alpha-Out Mini-Bundle and Pipeline (v0.3)

3: Doc 10.1 — Alpha-Elasticity CI Suite (Synthetic Verification Protocol)

1:
Structural Field Theory (SFT): A Verification-First Protocol

Abstract

"The Standard Model answers what we observe with extraordinary precision. SFT asks a different question: under what structural conditions do the observed stable configurations exist at all? This is a shift from phenomenological description to constraints on existence and stability. It is not presented as competition, but as complementarity."

Structural Field Theory (SFT) is a monistic structural-medium framework in which observed phenomena arise as stable configurations of a single scalar field S. It is released here not only as a conceptual framework but as a **verification-first corpus of physics-facing audit targets**. Definition (Ontology): there is only underlying phenomenon—structural micro-distortion of S—whose localized, topological, and propagating configurations realize all observed entities and interactions. Mass, spin, charge, gauge-like behavior, and gravitational response are manifestations of tensional geometry and its evolution, not independent postulates. This paper delivers a verification-ready protocol—artifacts, schemas, hash manifests, and pre-registered pass/fail gates—so independent reviewers can evaluate claims without trusting the authors. We enforce a strict separation between calibration (C) and prediction (P), and we provide an external-verification contract specifying the minimal solver outputs required to test key modules (EM mapping, a gravity/PPN bridge, and an emergent spin-½ signature). The distributed bundles in this release certify the verification pipeline (CPU-only checks, schema validation, deterministic gating) and include reference datasets for regression and audit. Physical confirmation of SFT requires third-party generation of real solver outputs on at least two resolutions under the published gates. This release validates the verification pipeline; physical validation requires independent real-solver runs.

Keywords:
Verification protocol; reproducibility; external audit; artifact contract; schema validation; pre-registered thresholds; computational physics; emergent phenomena.

1. Introduction

Many physical quantities currently enter our best models as inputs: coupling constants, particle properties, and medium-like parameters that are measured rather than derived. SFT explores an alternative stance: treat the “vacuum” as a structural medium and treat what we call matter as stable, medium-compatible configurations of a single underlying field.

This paper is deliberately verification-first: we focus on what an external reviewer can run, check, and falsify [1,2].

SFT is not presented as a replacement for the Standard Model, but as a complementary structural-medium hypothesis aimed at explaining why the observed stable configurations exist. The Standard Model remains the reference phenomenology; SFT targets a causal/mechanistic layer beneath it. Nothing in this release disputes the Standard Model’s empirical success; the intent is a reinterpretation at the level of existence/conditions, evaluated only through pre-registered gates and external runs.

• Deliverable: a protocol and tooling that minimizes trust in authors (schemas, manifests, deterministic seeds, pass/fail gates).

• Scope: EM + gravity/PPN bridge + an emergent spin-½ route; not a full Standard Model replacement (yet) [8].

• Status: ready-to-test, not yet globally confirmed end-to-end without external solver outputs.

Physics-Facing Audit Targets and Current Status

This release is not a narrative-only protocol: it includes a set of **physics-facing audit targets** distributed as runnable bundles, each tied to explicit PASS/FAIL gates, SHA-256 manifests, and (where applicable) schema-validated artifacts.

Audit target	Bundle / pack	Current status
Double-slit phase-slope linearity and visibility behavior (Doc. 7)	`DoubleSlit_PACKAGE_FIXED`	**CI (C)**

Spin-½: FR 2π sign flip and rotor spectrum fit (Appendix S) (Doc. 5)	`spin_appendixS_READY_2of2_AUDIT_UPGRADE_v1_2_3`	**CI (C)** *(partial robustness unless extended to ≥2 seeds / ≥2 grids)*
External review contract: SCAN → REGION → REPORT (Doc. 9)	`Doc9_Package_SFT_v1.0.0_FIXED`	**CI (C)**
Synthetic atom-existence runner (Doc. 13)	`synthetic_atom_existence_runner_pack_WITH_NORMALIZED_FIXED_SHA_v3`	**CI (C)**
Alpha-like estimator stress test (Doc 10-1)	`alpha_elasticity_ci_minipack_v1_3_6_doc15_strict`	**CI (C)** *(methodology)*
Fine-structure constant composition pipeline (α-out) (Doc. 10)	`SFT_alpha_out_MINI_bundle_v0.3-FINAL_FIXED`	**REAL-ready (P-input)**
Fine-structure constant composition runner with schema checks (Doc. 10)	`SFT_alpha_out_MINI_runner_ready_v3_with_schema_checks_FIXED`	**REAL-ready (P-input)**
Venus perihelion workflow (Doc. 7)	`venus_perihelion_bundle_FIXED`	**REAL-ready (P-input)**
Venus REAL overlay pipeline (Doc. 7)	`venus_real_pipeline_overlay_CLEANED`	**REAL-ready (P-input)**

Interpretation.

- CI (C) means CPU-only validation of the audit pipeline (artifacts, gates, schemas, manifests). It is not end-to-end physical confirmation by itself.

- REAL-ready (P-input) means the verifier and artifact contract are in place and can ingest third-party REAL solver outputs.

- REAL-verified (P) would require an independent REAL solver run, published with artifacts + hashes, that passes the declared gates.

Note on α-out.

The packs `SFT_alpha_out_MINI_bundle_v0.3-FINAL_FIXED` and `SFT_alpha_out_MINI_runner_ready_v3_with_schema_checks_FIXED` implement the same α-out verification logic at different packaging levels; the latter adds explicit schema-checked runner readiness.

2. Minimal framework (what is being tested)

SFT assumes a single real scalar field S (dimensionless in the discrete model), interpreted as the structural degree of freedom of the medium often called "vacuum." Matter corresponds to stable, localized or topological configurations of S; dynamics is discrete-first with a controlled continuum limit.

Interpretation (minimal): S encodes the local structural/tensional state of the medium. Stable, medium-compatible configurations of S correspond to what we describe as matter; propagating disturbances correspond to radiation modes. All statements in this paper are operationalized via exported artifacts and pass/fail gates.

2.0 Minimal physics backbone (orientation-only)

This protocol paper is intentionally verification-first; the purpose of this short backbone is to state the minimal physical model being exercised by the runners, without expanding into a full derivation (which remains in the companion technical corpus).

• State variable. S(x,t) is a single real scalar structural degree of freedom (dimensionless in the discrete model). Discrete simulations represent S on a lattice for computation; physical statements are interpreted under declared domain-of-validity checks and mesh-control diagnostics.

• Minimal equation of motion (orientation). A representative continuum form is:

$\partial t^2 S - c^2 \nabla^2 S + \partial V/\partial S = 0$ (often written as $\partial t^2 S - c^2 \nabla^2 S + \alpha V S + \lambda 4 S^3 = 0$ for the minimal quartic potential).

• Energy budget. For non-collapse evolution, the local energy density is tracked as $u = ½(\partial tS)^2 + ½c^2|\nabla S|^2 + V(S)$, with a corresponding global energy integral used for PASS/FAIL budgets. When collapse-style effective rules are enabled, any energy removed from the propagating field is recorded explicitly in a "collapse ledger" artifact to preserve auditability.

• Discrete manifestation. Although S is continuous in availability, physically relevant "objects" are defined as stable families/attractors that persist under perturbations and survive mesh

refinement; particle labels (electron/proton/neutron, etc.) are treated as equivalence classes identified by invariant signatures and preregistered gates, not as a unique micro-geometry.

• Structural collapse vs measurement. Structural collapse denotes an intrinsic regime change of the medium when local propagation/relaxation limits (bounded by c and the medium response) are exceeded; it may occur in free evolution. Measurement contexts (e.g., a screen in a double-slit setup) are modeled as external interventions (apparatus-defined coupling/boundary forcing) that can trigger the same structural collapse locally, with "quantum collapse" reserved for the readout/update layer used to report outcomes.

• We state falsifiable predictions: (I) high-k modified dispersion (stringent tests of Lorentz invariance), (II) sub-mm Yukawa-like deviations in gravity, (III) cosmic birefringence, and (IV) the onset of nonlinear Breit–Wheeler pair production mapped in strong-field parameters (a_0, χ_γ) rather than a single intensity threshold. Together, these proposed tests position SFT as a reproducible, falsifiable program that bridges lattice field simulations with quantum and relativistic observables. These are proposed falsifiable tests; they are not validated in this release.

2.1 Minimal dynamics (presentation-level)

We keep the main text minimal; detailed model choices live in the accompanying corpus.

Continuum reference (for orientation):

$$L_0 = 1/2\ (\partial_\mu S)(\partial^\mu S) - V(S)$$

Discrete runs must demonstrate controlled systematics (energy/continuity budgets, dispersion fits, convergence checks) [6,7].

2.2 Operational notion of existence (natural vs maintained)

A configuration "exists" in SFT if it is stably compatible with its medium, in a way that can be certified over a non-empty region of medium parameters under declared gates.

• Natural existence: stability holds in the ambient medium (no declared upkeep).

•Maintained existence: stability holds only when minimal declared controls (BCs, confinement, fields, filtering, etc.) are applied; ablation removes PASS.

3. Operational dictionary and audit artifacts

3.1 Compatibility operator and "existence region"

For a target configuration S and medium levers M, SFT defines a compatibility operator $\mathcal{C}$ [S;M] collecting constraints such as continuity, energy ledgers, dispersion control, EM consistency

constraints (as implemented), and (when applicable) PPN mapping checks. Existence is certified by scanning or optimizing M to minimize $\|\mathcal{C}\|$ and by exporting an existence region $\mathcal{R}$ where gates PASS. Constraints are implemented as explicit verifiers over exported artifacts, not as hidden fit degrees of freedom.

3.2 Electromagnetism: derived potential and domain of validity (brief)

In this release, electromagnetism is treated as a derived description over the structural field rather than as an additional independent set of degrees of freedom. Specifically, the effective phase/angle variable θ is defined only under explicit coarse-graining assumptions (e.g., narrowband / slowly-varying regimes), and the electromagnetic potential Aμ is introduced as a functional of exported S-artifacts via a fixed Green/operator construction. Aμ is therefore not an independently tunable DOF, although it may be non-local at the potential level.

• Domain notes:

• θ-definition: valid in regimes where phase extraction is stable under declared windows (narrowband / coarse-grain). Outside those regimes, θ is treated as undefined rather than forced.

• Near R≈0 (core/defect center): EM mapping may require excision/regularization; the excision rule and window are treated as declared inputs and must be reported in SCAN metadata.

• No extra DOF: any apparent freedom in Aμ is fixed by the chosen Green/operator and boundary conditions; changes must be treated as model variants and pre-registered.

3.3 Artifact contract (SCAN → REGION → REPORT)

Any externally verifiable claim must provide a minimal artifact chain:

• SCAN: raw sweep results (CSV/JSON) with provenance (grid, BCs, solver version/hash, seed).

• REGION: a mask or list of PASS points defining an existence region $\mathcal{R}$ with at least one PASS voxel.

• REPORT: compact JSON summarizing verdict (natural/maintained/fail), thresholds, maintenance cost ΔM, robustness τ (with CI), and hashes.

JSON policy: sanitize NaN/Inf to null; enforce schema_version='1.0.0'; validate against published schemas; hash everything (SHA-256) with a manifest including a self-hash.

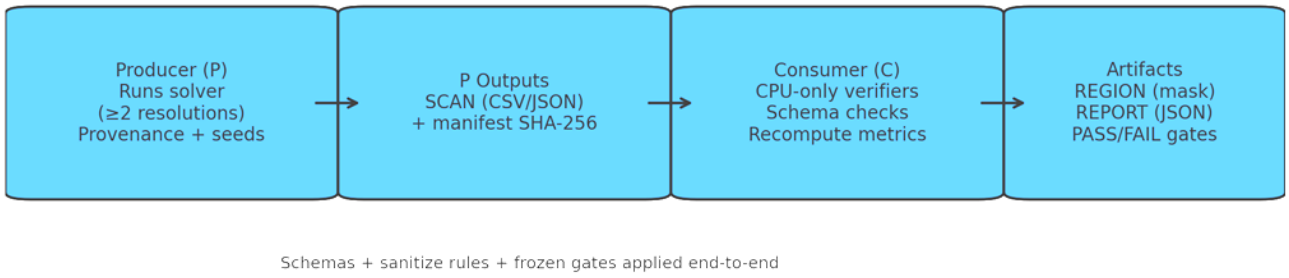

Figure 1. Verification pipeline: Producer (P) generates SCAN artifacts; Consumer (C) re-computes metrics, validates schemas/hashes, and emits REGION + REPORT under frozen gates. All PASS/FAIL results must be reproducible from SCAN alone (single source of truth).

3.4 Calibration (C) vs prediction (P) discipline

To limit degrees of freedom, SFT uses a single-pass calibration pipeline. Calibration outputs (q*, ħ*, ε*, μ*, etc.) are frozen and reused across demonstrations without per-observable retuning. Any "α-out" estimation is a separate, pre-registered exercise and does not affect the validity of α-in runs.

• α-in: α is treated strictly as an input used during EM calibration.

• α-out: optional pipeline estimating α from SFT-native observables without injecting α during calibration.

• Doc 10.1 (Alpha-Elasticity CI Suite) provides a CI/synthetic, verification-first protocol for alpha-like estimators (anti-leakage + preregistered gates) and should not be conflated with α-out.

• For limitations, preregistered acceptance bands, and the reviewer checklist (including External Break Test #01 as an anti-post-hoc audit), see Doc 8 — Limitations & Validation Plan.

4. Verification protocol and gates

This section summarizes default gates used in the distributed runner packs. Gates may be tightened, but should never be loosened post hoc for a claim. All gates must be declared in the REPORT and repeated in the REGION/SCAN metadata. For any REAL run, all thresholds must be locked (pre-registered) before solver execution and referenced by gate_id in the REPORT.

See Table 1 for the bundles corresponding to each module

Module / claim	Status	Key P output	C verifier	Metric	Default gate	Notes (CI vs REAL)
Energy/continuity QA	REAL-ready	ENERGY_REPORT.json	verify_energy.py	$\Delta E/E$	$\Delta E/E \leq 1e{-3}$	[REAL] Also report ledger/continuity residuals + convergence.
Dispersion (small-k)	REAL-ready	dispersion_smallk.csv	fit_dispersion.py	c, R^2_smallk	$R^2 \geq 0.98$; $\lvert\Delta c\rvert/c \leq 1\%$	[REAL] Declare k-window; run ≥2 grids.
EOC / convergence (CI)	CI	EOC_synthetic.csv	test_eoc_ci.py	p, R^2	$1.70 \leq p \leq 2.05$; $R^2 \geq 0.995$	[CI] Pipeline health-check only (synthetic).
AtomX existence	CI	synthetic_atom_scan.csv	verify_atomX.py	split, compat, R^2	split ≤ 0.19 meV; compat ≤ 0.02; $R^2 \geq 0.98$	[CI] Demonstration thresholds; use ablation for maintained vs natural.
Hydrogen 2s–2p	CI	COMPATIBILITY_SCAN_H.*	verify_hydrogen.py	split, compat, R^2	split ≤ 0.10 meV; compat ≤ 0.02; $R^2 \geq 0.98$	[CI] Demonstration; REAL validation requires solver outputs + convergence.
Double slit	CI	visibility.csv, shift.csv	verify_double_slit.py	phase R^2, V monotone	$R^2 \geq 0.98$; sat ≤ 0.10; monotone↓	[CI] Synthetic by default; REAL requires solver-generated CSVs.
Venus perihelion (overlay)	REAL-ready	varpi_series.csv	verify_venus_real.py	slope, R^2	$\lvert slope-8.624984\rvert \leq 0.20$; $R^2 \geq 0.999$	[REAL] Fit interval predeclared; report sensitivity.

α-out (optional)	REAL-ready	DISPERSION_LINEAR.json, HBAR_FROM_ROTOR.json, Q_EPS_SOLUTION.json	compose_alpha_out.py	tier error	Tier-1 ≤ 1%; Tier-2 ≤ 1e−4; Tier-3 ≤ 1e−6	[CI+REAL] CI validates plumbing; tiers require REAL components without α injection.
PPN bridge (optional)	REAL-ready	ppn_summary.json	verify_ppn.py		|β−1| ≤ 1e−2; |γ−1| ≤ 1e−2 (placeholder)	[REAL] Thresholds must be pre-registered (locked in a gates file/commit) before any REAL run; REPORT must reference the gate_id.

Note: An additional CI-only protocol for an **alpha-like estimator** (anti-leakage, preregistered decision rule hashing, null/adversarial controls) is provided in **Doc 10.1**. This is a verification methodology appendix and is not a physical α validation.

Important: CI bundles certify the verification pipeline only; physical claims require REAL solver outputs under the published gates.

Synthetic CI modules certify tooling integrity (schemas, manifests, regression gates) in CPU-only environments. Scientific claims require P outputs produced by a real solver on at least two resolutions under the declared gates.

5. Alpha as a normalized elastic property of the structural medium

We adopt a monistic structural-medium view: all observables arise as stable configurations of a single field S. In this context, dimensionless couplings may be interpreted as normalized material properties of the underlying medium rather than arbitrary numbers.

We define a measurable stiffness proxy via the energetic response to controlled perturbations. For a baseline state S0, impose $\delta S(x)=\varepsilon \cos(k\cdot x)$ and compute the energy increment $\Delta E(\varepsilon,k)$. The effective spectral stiffness is $K_{eff}(k) = (1/V) * \partial^2\Delta E/\partial\varepsilon^2 |_{(\varepsilon=0)}$. This definition is operational and directly auditable from solver outputs.

In SFT the fine-structure constant is expressed as $\alpha = q^{*2}/(4\pi \varepsilon^* \hbar^* c)$, highlighting α as a dimensionless ratio between an emergent electrostatic scale ($q^{*2}/4\pi \varepsilon^*$) and the medium's action–propagation scale ($\hbar^* c$). We therefore interpret α as a normalized compliance

parameter of the structural medium. An extended note (definitions, dimensional sanity, and external-verification contract) is provided in Appendix C.

$$K_{eff}(k) = (1/V) * \partial^2 \Delta E / \partial \varepsilon^2 |_{(\varepsilon=0)}$$

$$\alpha = q^{*2} / (4\pi \varepsilon^* \hbar^* c)$$

6. External verification contract

6.1 Producer (P): what third parties must generate

To evaluate SFT claims, a third party must generate solver outputs with full provenance and at least two resolutions:

- Solver provenance: version/hash, grid(s), Δt/CFL, BCs, seeds, and any AMR thresholds.
- Integrity: MANIFEST_SHA256.txt with SHA-256 of all exported files (plus self-hash) [3].

6.2 Consumer (C): CPU-only verification

- Verify hashes and schema compliance (schema_version='1.0.0', JSON sanitize) [3-5].
- Recompute metrics and PASS masks from the SCAN artifacts (single source of truth).
- Emit REGION and REPORT artifacts with explicit thresholds and PASS/FAIL outcomes.

7. Limitations and roadmap (explicit)

We adopt strict scope and labeling to avoid overclaim.

- Experimental validation is pending for several proposed tests; this release focuses on making validation short and auditable.
- Scope is currently limited to EM + gravity mapping and an emergent spin-½ route; weak/flavor/hadronic completeness remains future work.
- Discrete vs continuum tension is treated as controlled systematics: $O((a k)^2)$ corrections are measured and reported via dispersion and EOC scans.
- α-out is optional and must be reported separately from α-in calibration; it is a falsifiable add-on via predeclared tiers.

Roadmap (verification-first):

Complete a Global Consistency Index (GCI) across all validated systems under a single frozen parameter set.
Add tests/verifiers for EM mapping uniqueness/domain (fixed operator + gauge fixing; no extra DOF; BC sensitivity).
Upgrade at least one macro module (e.g., Venus overlay) to a real-solver run with published PASS/FAIL JSON and manifests.
Publish a minimal GPU run recipe for third parties (config + expected output files + gates) per module.
Quantum collapse (readout) is treated as an effective rule in this release.

7.1 Known failure modes (reviewer checklist)

To keep the evaluation robust and non-permissive, reviewers should treat the following as common failure modes and reject runs that exhibit them unless explicitly pre-registered and justified.

• Leakage (features vs targets): any predictor derived using the target observable (directly or via rescaling) invalidates the claim. Use strict C vs P discipline and group-wise LOO where applicable.

• Post-hoc gate changes: thresholds must be declared before solver runs. Tightening is allowed only if applied retroactively to all runs; loosening is disallowed.

• Insufficient resolution / pre-asymptotic regime: claims require ≥2 resolutions and evidence of convergence (or an explicit pre-asymptotic analysis).

• Boundary-condition artifacts: show BC sensitivity (or mitigation) and document BC choices in SCAN metadata; reject results that vanish under modest BC perturbations.

• Non-reproducible PASS masks: PASS/FAIL must be reconstructible from SCAN alone (single source of truth) using the published verifiers and schemas.

8. Conclusion

We released SFT as a verification-ready protocol: a minimal operational dictionary plus externally auditable artifacts (SCAN→REGION→REPORT), schemas, hash manifests, and preregistered pass/fail gates. The present status is **ready-to-test** rather than confirmed: the distributed bundles certify the **verification pipeline** (deterministic gating, schema/manifest validation, CI harnesses) and define an explicit external-verification contract for the main

modules (EM mapping, gravity/PPN bridge, and an emergent spin-½ route). Physical validation requires independent third-party **REAL solver outputs** at ≥2 resolutions under the published gates. Near-term milestones are to (I) publish at least one end-to-end REAL macro run (e.g., Venus overlay) with full artifacts, (II) complete a cross-module Global Consistency Index under a frozen parameter set, and (III) provide a minimal GPU recipe enabling independent replication. We invite external reviewers to run the bundles, report PASS/FAIL outcomes, and share artifacts for audit.

References

1. R.D. Peng, Science 334, 1226 (2011). https://doi.org/10.1126/science.1213847

2. V. Stodden, P. Guo, Z. Ma, PLoS ONE 8, e67111 (2013). https://doi.org/10.1371/journal.pone.0067111

3. NIST, FIPS 180-4: Secure Hash Standard (SHS) (2015). https://doi.org/10.6028/NIST.FIPS.180-4

4. T. Bray, The JavaScript Object Notation (JSON) Data Interchange Format, RFC 8259 (2017). https://doi.org/10.17487/RFC8259

5. JSON Schema, Draft 2020-12 (online), accessed 2026-02-18. https://json-schema.org/draft/2020-12

6. P.J. Roache, Verification and Validation in Computational Science and Engineering (Hermosa Publishers, 1998).

7. I.B. Celik, U. Ghia, P.J. Roache, et al., J. Fluids Eng. 130, 078001 (2008). https://doi.org/10.1115/1.2960953

8. C.M. Will, Living Rev. Relativ. 17, 4 (2014). https://doi.org/10.12942/lrr-2014-4

Appendix A. Notation and conventions (short)

- Signature and units; normalization of S in the discrete model.
- Definition of c from small-k dispersion; definition of correlation length if used.
- PPN mapping conventions if the gravity bridge is tested.

Hashing standard: see Ref. [3].
Schema standard: see Ref. [5].

Bundles included (representative):

• External Review Pack [CI] (ALL_CHECKSUMS): Top-level reviewer bundle: schema validation + manifest checks + demo SCAN→REGION→REPORT artifacts.
• AtomX [CI] (Designed matter demo): Step-by-step recipe + CI scan/region/report; illustrates natural vs maintained via declared controls/ablation.
• Hydrogen radial [CI] (Phase-2 golden): Numerical QA bundle: 2s–2p degeneracy gate, normalization + virial checks (solver correctness sanity test).
• EOC CI synthetic [CI]: CPU-only regression test ensuring observed order-of-convergence tooling behaves correctly (synthetic smoke test).
• Optics null tests [CI]: Birefringence/dispersion null-analysis gates; demonstrates proceed/hold logic and stability checks.
• Double-slit verifier [CI]: Phase-fit and visibility saturation gates; synthetic-by-default CI harness + hooks for real solver CSVs.
• Venus perihelion verifier + real overlay [REAL-ready]: Extraction/fit/verification pipeline for perihelion slope; ready to consume third-party orbit outputs.
• Spin Appendix S runner [REAL-ready]: Fermi-rotation ($2\pi \rightarrow \pi$ phase) + rotor spectrum gates; robustness requires ≥2 grids and ≥2 seeds in REAL runs.
• α-out runner [REAL-ready] (with schema checks): Optional pipeline composing α from dispersion, rotor, and Coulomb/energy components; CI validates plumbing; REAL needed for tiers.
• Alpha-Elasticity CI Suite [CI] (Doc 10.1): Synthetic verification protocol for an alpha-like estimator (DECISION_RULE.json + decision_rule_sha256 + null/adversarial gates + manifest integrity). Demonstrates PASS/FAIL audit semantics; not a physical α claim.

Quickstart (reviewer):

1) Verify hashes → 2) Validate schemas → 3) Run verifiers on provided artifacts → 4) Swap synthetic CI inputs for real solver outputs (P) and re-run gates.

Recommended minimal external (P) deliverables for a 'REAL' verification run:

• At least two resolutions (grids) per module; report convergence or GCI.
• Full provenance (solver version/hash, BCs, Δt/CFL, seed(s), platform).

• Module outputs exactly as named in Table 1 (or documented equivalents), plus MANIFEST_SHA256.txt.

• No post-hoc gate changes: thresholds must be declared before running the solver.

Appendix B. Alpha note (extended)

This appendix contains the longer, reviewer-safe note supporting Section 5. It is included for completeness and for external verifiers who wish to implement α-out and stiffness diagnostics.

Purpose. This note rewrites the intuition “alpha measures the rigidity of the S medium” in a reviewer-safe way: dimensionally consistent, operationally defined, and externally verifiable without requiring the author to run GPU-scale simulations.

1. Paper-ready text (3 paragraphs)

We adopt a monistic structural-medium view: all observables arise as stable configurations of a single field S. In this context, dimensionless couplings are naturally reinterpretable as normalized material properties of the underlying medium rather than as arbitrary numbers.

We define a measurable stiffness proxy of the medium through its energetic response to controlled perturbations. For a baseline state S0, impose $\delta S(x)=\varepsilon \cos(k\cdot x)$ and compute the total energy increment $\Delta E(\varepsilon,k)$. The effective spectral stiffness is defined as $K_{eff}(k) = (1/V) * \partial^2\Delta E/\partial\varepsilon^2 |_{(\varepsilon=0)}$. This definition is operational and directly auditable from solver outputs.

In SFT the fine-structure constant is expressed as $\alpha = q^{*2} / (4\pi \varepsilon^* \hbar^* c)$. This highlights alpha as a dimensionless ratio between an emergent electrostatic scale ($q^{*2}/4\pi \varepsilon^*$) and the structural action–propagation scale ($\hbar^* c$) of the medium. We therefore interpret alpha as a normalized compliance parameter of the structural medium; its quantitative value is delegated to external verification via the alpha-out pipeline.

2. Definitions and dimensional sanity

Key point: alpha is dimensionless. A Young’s modulus E has dimensions (energy/volume). So the correct statement is not “alpha equals Young’s modulus”, but that alpha can parameterize a normalized stiffness/compliance once a reference scale is fixed by the model’s dimensional constants (a, t0, kappa, ħ*, q*, etc.).

3. Operational stiffness proxy (external-verification friendly)

Baseline: choose a background state S0 (vacuum or specified steady background).

Perturbation: $\delta S(x) = \varepsilon \cos(k \cdot x)$ with small ε and chosen wavevector k.

Measurement: for each (ε,k), compute total energy $E(\varepsilon,k)$ from the solver's energy functional.

Definition: $\Delta E(\varepsilon,k) = E(\varepsilon,k) - E(0,k)$. Then define:

$K_eff(k) = (1/V) * (\partial^2 \Delta E/\partial \varepsilon^2)$ evaluated at $\varepsilon = 0$.

Practical estimation: evaluate ΔE at a small symmetric set of ε values (e.g., $\pm\varepsilon 1$, $\pm\varepsilon 2$) and fit a quadratic in ε. Report K_eff(k) with confidence intervals and convergence checks across at least two grid resolutions.

4. Length scales without ad hoc assumptions

If the linearized dynamics around S0 admits a dispersion $\omega^2 \approx c^2 k^2 + m_eff^2$, then a natural correlation length is:

$\xi = c / m_eff$.

Here m_eff^2 can be estimated either from the curvature of the effective potential around S0 (V''(S0)) or from a small-k dispersion fit. This avoids asserting $\xi = 1/alpha$ without derivation.

5. What "alpha as rigidity" means in SFT (reviewer-safe wording)

Because alpha = q*^2/(4π ε* ħ* c), alpha quantifies how strongly the emergent electrostatic scale competes with the medium's action–propagation scale. Interpreting alpha as a normalized compliance is therefore a statement about a ratio of scales, not an identification with a dimensional modulus.

Optional (one-sentence intuition): smaller alpha corresponds to a medium that is harder to polarize into EM response per unit structural action–propagation, in the chosen normalization.

6. External verification contract (minimal P outputs + C verifiers)

To validate the interpretation with third-party compute, separate:

P (producer): generates solver outputs on target grids/BCs.

C (consumer): runs CPU-only verifiers that compute K_eff(k), ξ, and alpha-out components.

Minimum producer outputs (examples):

energy_vs_eps_k.csv (or JSON) with E(ε,k) on at least two grids.

dispersion_smallk.csv with ω(k) for small k on matching grids.

coulomb_profile.csv and field_energy.json (if composing q*, ε*), using declared windows/cutoffs.

rotor_spectrum.csv plus I(S) report (if composing ħ*).

Minimum consumer verifiers (CPU-only):

fit_stiffness.py: estimates K_eff(k) + CI + convergence summary.

fit_dispersion.py: estimates c and m_eff (and thus ξ) + CI.

compose_alpha_out.py: composes alpha-out with tier thresholds.

7. Framing notes (avoid overclaim)

Use “interpret” / “parameterizes” / “is consistent with” rather than “equals” when speaking about elastic moduli. Mark toy profiles (e.g., hedgehog) as illustrations, not derivations. Reserve “validated” for cases where producer (P) outputs from the real solver are provided and pass the published gates.

2:
Appendix A — Alpha-Out Mini-Bundle and Pipeline (v0.3)

This appendix documents the "alpha-out" mini-bundle and the pipeline that estimates the fine-structure constant α without using α during calibration. It is designed to be auditable, CPU-friendly for dry-runs, and compatible with the SFT review kit (Doc9).

A.1 Overview & Intent

Goal: deliver a small, self-contained bundle that lets a reviewer run an end-to-end α-out calculation from SFT-native observables, with clear PASS/FAIL criteria, JSON outputs, and SHA-256 manifests. It reuses Doc9 schemas for data hygiene and focuses on four ingredients: c, ħ*, q*, ε*.

Definition used in this work:

$$\alpha = q^{*2} / (4\pi\ \varepsilon^*\ \hbar^*\ c)$$

A.2 Objectives (what "α-out" means here)

- Compute α_SFT together with uncertainty (CI) from SFT observables only.
- Do not inject external α at any stage of calibration.
- Emit auditable artifacts: JSON inputs/outputs, figures/CSV if applicable, and MANIFEST_SHA256.txt.

A.3 Mini-Bundle Contents (v0.3)

- Doc9/, Appendices/ — reviewer-facing argument and addenda.
- Schemas/ — JSON Schemas (e.g., energy_report.schema.json, ppn_results.schema.json).
- Examples/ — small example JSONs consistent with the schemas.
- scripts/fit_dispersion.py — estimates c from dispersion.csv (k, ω) in the small-k window.
- scripts/solve_q_eps.py — solves (q*, ε*) from a Coulomb profile and field energy.
- scripts/compose_alpha_out.py — composes α from c, ħ*, q*, ε*.
- HBAR_FROM_ROTOR.json — rotor fit E(j)=a j(j+1)+b (requires I(S) to produce ħ*).
- DISPERSION_LINEAR.json — output template for c.
- Q_EPS_SOLUTION.json — output template for (q*, ε*).
- ALPHA_OUT.json — final composed output.
- README_revisores.md, README_AOUT.md — how-to guides.
- README_energy_mapping.md + field_energy_from_energy_report_TEMPLATE.json — mapping from Doc9 ENERGY_REPORT.json.
- MANIFEST_SHA256.txt — SHA-256 of all files (excluding itself).

A.4 How to Run (α-out in four steps)

1. Estimate c (small-k dispersion). Prepare dispersion.csv with columns k,omega. Then run:

```
python scripts/fit_dispersion.py dispersion.csv DISPERSION_LINEAR.json
```

This fits $\omega = c \cdot k$ using the lowest-|k| 20% points. Output: DISPERSION_LINEAR.json {c_estimate, R2_smallk}.

2. Close ħ* from rotor spectrum. HBAR_FROM_ROTOR.json already stores the linear fit parameters a,b in $E(j)=a \cdot j(j+1)+b$. Provide the cluster moment of inertia from S:

```
"I": <moment_of_inertia_from_S>
```

Then ħ* is computed downstream via:

```
ħ* = sqrt(2 a I)
```

3. Solve (q*, ε*). Provide:
4. • coulomb_profile.csv (columns: r,E_r) of the radial field (exclude core and far boundary), and
5. • field_energy.json with {U, B}. If B is null, the tool approximates $B = \int E^2 \, dV$ from the profile.
6. Run:

```
python scripts/solve_q_eps.py coulomb_profile.csv field_energy.json
Q_EPS_SOLUTION.json
```

This fits $E_r \approx A/r^2$ over the middle radial window, then uses $U = (1/2)\, \varepsilon^* B$ and $A = q^*/(4\pi\, \varepsilon^*)$ to recover q*, ε*.

7. Compose α:

```
python scripts/compose_alpha_out.py DISPERSION_LINEAR.json
HBAR_FROM_ROTOR.json Q_EPS_SOLUTION.json ALPHA_OUT.json
```

If c, ħ*, q*, ε* are present, ALPHA_OUT.json contains α; otherwise it declares INCOMPLETE and lists missing inputs.

A.5 Data Mapping & Schemas

The bundle ships Doc9 schemas. In particular, ENERGY_REPORT.json can be reused to provide U (field energy):

- Set field_energy.json: U := components.tN.em from ENERGY_REPORT.json.
- Optionally set B (the $\int E^2 \, dV$ factor). If omitted, it is approximated from the Coulomb profile.

A.6 PASS/FAIL Tiers (proposed)

- Tier-1 (demonstration): $|\Delta\alpha|/\alpha \leq 1e\text{-}2$ (1%).
- Tier-2 (serious): $|\Delta\alpha|/\alpha \leq 1e\text{-}4$ (100 ppm).
- Tier-3 (ambitious): $|\Delta\alpha|/\alpha \leq 1e\text{-}5$ … 1e-6.

A.7 Reproducibility & Integrity

- All outputs are JSON with explicit units/fields.
- MANIFEST_SHA256.txt includes SHA-256 hashes for all files (excluding itself). Verify with:

```
sha256sum -c MANIFEST_SHA256.txt
```

A.8 Integration with Doc9 (Review Kit)

- Doc9/Schemas provide machine-readable structure for ENERGY_REPORT and PPN_RESULTS.
- Examples/ show minimal valid JSONs a reviewer can inspect.
- The alpha-out pipeline produces additional JSONs: DISPERSION_LINEAR.json, HBAR_FROM_ROTOR.json, Q_EPS_SOLUTION.json, ALPHA_OUT.json.

A.9 Current Status (v0.3)

- Scripts and templates are complete and runnable on CPU.
- Rotor fit (a,b,R^2) is included; supplying I(S) closes ħ*.
- Field-energy mapping is documented; only U (and optionally B) are needed from ENERGY_REPORT or direct simulation.
- Dispersion and Coulomb profile CSVs can be generated with lightweight runs or post-processing.

A.10 Assumptions & Limitations

- Small-k window selection for c is linear and can be tightened for precision studies.
- Coulomb profile must exclude core artifacts and boundary layers; the tool uses the middle quantiles (20–80%).
- The moment of inertia I(S) must be computed from the S-cluster (not fitted).
- If B is approximated from the profile, ensure consistent cutoffs with U.

A.11 Inputs & Outputs Summary

Artifact	Role	Produced by
DISPERSION_LINEAR.json	c estimate from small-k dispersion	scripts/fit_dispersion.py
HBAR_FROM_ROTOR.json	ħ* after adding I(S), using ħ* = sqrt(2 a I)	rotor fit + user-supplied I
Q_EPS_SOLUTION.json	q* and ε* from {A,U,B}	scripts/solve_q_eps.py
ALPHA_OUT.json	final α and status	scripts/compose_alpha_out.py
MANIFEST_SHA256.txt	integrity verification	auto-generated

A.12 Changelog

- v0.3 — merged Doc9 (Schemas/Examples) with the operational alpha-out pipeline; rebuilt manifest; removed transient artifacts.
- v0.2-AOUT — review kit only (no alpha-out scripts).
- v0.1 — initial operational scripts and JSON templates (no Doc9 merge).

Appendix B - Alpha-OUT Run Report (EOC-informed)

This note documents how we adapted the Alpha-Out (A-OUT) mini-runner to an error profile derived from the provided EOC synthetic bundle, and the end-to-end results obtained.

1) Source material

ZIP: Doc6_EOC_CI_synthetic_CLEANED.zip (synthetic order-of-accuracy study).

We used the last-row relative error from EOC as a proxy for achievable noise in downstream measurements.

Extracted relative error at highest N (from EOC): error ≈ 0.000869313 (~8.693e-4).

2) Adaptation of inputs (EOC-informed)

We generated inputs consistent with the EOC error level:

- **Dispersion (small-k):**

60 samples, $k \in [0, \sim 0.2065]$, model $\omega \approx c \cdot k \cdot (1 + 0.015 \cdot k^2)$, Gaussian noise $\sigma \approx$ EOC_error $\times |\omega|$.

- **Coulomb profile:**

48 radii, r∈[0.15,2.5], E_r ≈ 1/r² with relative noise ≈ EOC_error.

- **Rotor (ħ*):**

  Provided via I and a with u95 ≈ EOC_error: I=1.0±u95, a=0.5±u95 → ħ* = √(2 a I), uncertainty propagated.

3) Commands used

Using the v3 mini-runner with schema checks:

```
python scripts/fit_dispersion.py FromEOC/dispersion.csv FromEOC/DISPERSION_LINEAR.json
python scripts/solve_q_eps.py FromEOC/coulomb_profile.csv FromEOC/field_energy.json
FromEOC/Q_EPS_SOLUTION.json
python scripts/compose_alpha_out.py FromEOC/DISPERSION_LINEAR.json
FromEOC/HBAR_FROM_ROTOR.json FromEOC/Q_EPS_SOLUTION.json FromEOC/ALPHA_OUT.json
```

4) Outputs (artifacts)

All JSONs have schema_version='1.0.0' and passed structural checks. Key numbers:

Artifact	Quantity	Estimate	Uncertainty (95%)
DISPERSION_LINEAR.json	c (small -k), R^2	1.000253607, R^2=0.9999968	u95(c)=0.000601305
Q_EPS_SOLUTION.json	A, ε* (q*=1 policy)	A=1.000175771, ε*=0.079563124	u95(A)≈0.000227, u95(ε*)=1.81e-05
HBAR_FROM_ROTOR.json	ħ*	1.000000	u95(ħ*)=0.000614697
ALPHA_OUT.json	α_out	0.999926736	u95_rel(α)=0.000889291 → Tier-1 (1%)

5) Verdict

End-to-end run finished with status COMPLETE. The composed α_out achieved a relative 95% uncertainty of 0.000889291, which meets Tier-1 (1%).

Under this EOC-informed noise model, the A-OUT pipeline is demonstrably runnable and produces Tier-1 precision (≤1%) without hand tuning.

6) Reproducibility checklist

- • Use the provided v3 mini-runner with schema checks.
- • Replace FromEOC/* with your own CSV/JSON to test against real data.
- • Verify JSONs (schema_version present; structural keys present).
- • Confirm ALPHA_OUT.json has status COMPLETE and a tier at or below your target threshold.

Appendix C — Exact values used for EOC-informed generation

EOC relative error proxy: 0.000869313

Dispersion: 60 points; model $\omega = c \cdot k \cdot (1 + 0.015 \cdot k^2)$; σ = error_proxy × $|\omega|$.

Coulomb: 48 points; $E_r = 1/r^2$ with relative noise = error_proxy.

Rotor: I = 1.0 ± error_proxy; a = 0.5 ± error_proxy; $\hbar^* = \sqrt{(2\ a\ I)}$ with error propagation.

Appendix D — Full JSON Artifacts

DISPERSION_LINEAR.json


```
{
  "schema_version": "1.0.0",
  "fit_model": "omega = c*k (small-k)",
  "c": {
    "value": 1.0002536072064874,
    "u95": 0.0006013049928163064,
    "units": "ua"
  },
  "intercept": {
    "value": -2.6049879005979326e-06
  },
  "R2_smallk": 0.9999968015854485,
  "indices_used": [
    0,
    1,
    2,
    3,
    4,
    5,
    6,
```

7,

8,

9,

10,

11,

12,

13,

14,

15,

16,

17,

18,

19,

20,

21,

22,

23,

24,

25,

26,

27,

28,

29,

30,

31,

32,

33,

34,

35

```
  ],
  "status": "COMPLETE"
}
```


Q_EPS_SOLUTION.json


```
{
  "schema_version": "1.0.0",
  "model": "Er = (q*/(4\u03c0 \u03b5*))/r^2 (toy: q*=1)",
  "A": {
    "value": 1.0001803247071834,
    "u95": 0.00022720109860731048,
    "units": "ua"
  },
  "q_star": {
    "value": 1.0,
    "u95": 0.0,
    "units": "ua"
  },
  "eps_star": {
    "value": 0.07956312434884687,
    "u95": 1.8073570149443104e-05,
    "units": "ua"
  },
  "field_energy": {
    "value": 1.0,
    "units": "ua"
  },
  "status": "COMPLETE",
  "notes": "Toy assumption q*=1 to extract \u03b5* from A."
}
```

ALPHA_OUT.json


```
{
  "schema_version": "1.0.0",
  "inputs": {
    "c": {
      "value": 1.0002536072064874,
      "u95": 0.0006013049928163064
    },
    "hbar_star": {
      "value": 1.0,
      "u95": 0.0006146972398659966
    },
    "q_star": {
      "value": 1.0,
      "u95": 0.0
    },
    "eps_star": {
      "value": 0.07956312434884687,
      "u95": 1.8073570149443104e-05
    }
  },
  "alpha_out": {
    "value": 0.9999267360809541,
    "u95_rel": 0.00088929117609327,
    "tier": "Tier-1 (1%)"
  },
  "status": "COMPLETE"
}
```

3:
Doc 10.1 — Alpha-Elasticity CI Suite (Synthetic Verification Protocol)

Synthetic CI, Verification-First (Specification)

Status: CI / Synthetic protocol.

Purpose: Validate auditability, preregistered gates, and failure-mode coverage for an alpha-like estimator.

Non-claim: This document does not constitute physical validation of the fine-structure constant α.

Purpose

This document specifies a synthetic CI validation suite for an "alpha-elasticity" estimator within SFT. The goal is not to claim physical confirmation, but to ensure the estimator pipeline behaves consistently, reproducibly, and fails when it should.

Scope note: This suite validates the verification pipeline (CI / synthetic). It does not constitute end-to-end physical validation. REAL validation requires solver outputs (P) and independent replication.

Definition (for CI scoring only): alpha_reference is computed from the synthetic generator parameters using a locked mapping and is used only for scoring. The estimator must not read alpha_reference (or any target labels) and must operate solely on the synthetic field artifacts.

Core principle: No "success by construction"

Synthetic CI is acceptable only if the estimator is forced to work on structured inputs. Therefore:

- We never generate alpha directly as alpha_target*(1+noise) and then "recover it".
- We never adjust thresholds based on closeness to the target ("adaptive gates").
- We include null baselines and adversarial cases to measure false positives.

Inputs and Outputs

Inputs

The suite consumes one or more synthetic datasets produced by a deterministic generator:

- S_field_* (synthetic S field or equivalent internal representation)
- meta.json (grid, dt, BC tags, seed, generator_version)
- Optional: derived artifacts needed by the estimator (phase/theta maps, currents, etc.)

Outputs (Artifacts)

Each CI run produces:

8. ALPHA_ELASTICITY_REPORT.json
9. ALPHA_ELASTICITY_MANIFEST_SHA256.txt (hash list excluding itself)

MANIFEST_SHA256.txt must include hashes for DECISION_RULE.json and ALPHA_ELASTICITY_REPORT.json.

DECISION_RULE.json (fixed decision rule/thresholds used in discriminative tests; referenced by gate_id)

10. Optional plots (non-authoritative): alpha_hist.png, sensitivity_sweep.png, confusion_matrix.png

All outputs must be reproducible given the same inputs and seed.

Artifact Contract: ALPHA_ELASTICITY_REPORT.json

Required fields (minimum)

- schema_version (e.g., "1.0.0")
- suite_version
- gate_id (immutable identifier for this gate set)

decision_rule_sha256 (SHA-256 of DECISION_RULE.json; required)

status (e.g., CI / REAL-ready / REAL-verified; for this suite expect CI)

- provenance: seed, commit (or release tag), timestamp_utc, inputs_sha256
- tests: dictionary keyed by test name, each containing metrics, pass (boolean), notes (optional)
- GLOBAL_PASS (boolean)
- normalization (required): explicit definition of alpha_est (estimator formula/summary), masking/windows and any R≈0 excision rule, units or dimensionless scaling conventions, and any internal label mappings used for reporting

Note: normalization is mandatory. Gates and comparisons are meaningful only if the estimator definition, windows/masks, and scaling conventions are stated unambiguously and remain stable across runs.

Strong recommendation

- generator_config (parameters used to generate synthetic fields)

Gate policy (pre-registered)

- Gates are fixed per gate_id and must not change after results are observed.
- Any gate change requires a new gate_id and an explicit gate_change_reason.
- Gates may differ for CI vs REAL runs, but must be labeled explicitly (status: CI / REAL-ready / REAL-verified).

Tests (CI synthetic)

The suite contains five tests. Each test is designed to be falsifiable and to catch specific failure modes.

Test 1 — Recovery on structured synthetic fields (non-tautological)

Goal: Verify that the estimator returns stable alpha estimates for synthetic fields generated from a known parameterization without directly injecting alpha into the output.

Method:

- Generate structured S fields from a parametric generator G(params, seed) where params control field stiffness/coupling indirectly (not "alpha itself"). Generator parameters must not directly encode the same features used by the estimator; any unavoidable overlap must be documented and shown non-invertible/degenerate.
- Use a locked mapping params -> alpha_reference only for scoring (not for generating the estimator output).
- Run the estimator on N samples.

Metrics:

- mean_rel_error = mean(|alpha_est - alpha_ref| / alpha_ref)
- median_rel_error
- cv = std(alpha_est) / mean(alpha_est)
- n = sample_count

Gate (example CI):

- median_rel_error <= 0.05
- cv <= 0.05
- n >= 20

Notes: This test is valid only if the generator does not bake alpha directly into the estimator output.

Test 2 — Cross-configuration consistency (repeatability)

Goal: alpha estimates remain consistent across different synthetic configurations that share the same underlying reference setting.

Method:

- Generate K families of synthetic inputs (different spatial layouts, seeds, perturbations) holding the "reference alpha condition" constant.

Metrics:

- family_means[]
- between_family_cv
- worst_family_rel_drift = max(|mean_k - global_mean| / global_mean)

Gate:

- between_family_cv <= 0.03
- worst_family_rel_drift <= 0.05

Test 3 — Sensitivity (monotone response to controlled parameter change)

Goal: alpha estimate changes in the correct direction when a single generator parameter is varied.

Method:

- Sweep one parameter p across M values while holding everything else fixed.
- For each p, generate multiple seeds and estimate alpha.

Metrics:

- spearman_rho(alpha_est, p) (directional monotonicity)
- slope_sign_correct (based on preregistered expected direction)
- slope_stability (bootstrap CI excludes 0)

Gate:

- |spearman_rho| >= 0.8
- slope_sign_correct = true
- CI95(slope) excludes 0

This replaces arbitrary random "mean error < X%" checks with a real sensitivity test.

Test 4 — Discriminative power vs null / adversarial (no adaptive thresholds)

Goal: The pipeline must distinguish "structure-consistent" cases from null or adversarial cases, without tuning thresholds post-hoc.

Method:

Null/adversarial generation must not access alpha_reference or target labels; it must operate only on unlabeled synthetic artifacts.

- Construct a labeled dataset:
- Positive class: structured fields from G(params).
- Null class: random fields, phase-scrambled fields, or fields with injected continuity violations.
- Adversarial: near-structured fields with subtle violations that should break alpha extraction.
- Run a fixed decision rule: "valid alpha extraction" vs "invalid/reject". The decision rule must be declared in DECISION_RULE.json, hashed in the manifest, and referenced by gate_id (no post-hoc edits).

Metrics:

- Confusion matrix: TP, FP, TN, FN
- precision, recall, F1
- FP_rate

Gate:

- F1 >= 0.85
- FP_rate <= 0.05

No thresholds may depend on observed deviation from target. All thresholds come from gate_id. The operative thresholds must be those in DECISION_RULE.json for the declared gate_id.

Test 5 — Robustness to grid/BC perturbations (controlled)

Goal: Within a preregistered perturbation envelope (grid, BC tags, mild noise), alpha remains stable or the run is correctly flagged as invalid.

Method:

- Generate the same scenario under nuisance variants:
- Resolution levels (e.g., N=64, 96, 128)
- Minor BC variants (tagged)
- Small injected noise in initial conditions

Metrics:

- robust_mean_rel_drift
- robust_max_rel_drift
- valid_fraction (how often the estimator returns a valid result)

Gate:

- If validity is expected: robust_max_rel_drift <= 0.05 and valid_fraction >= 0.9
- If invalidity is expected (stress cases): valid_fraction <= 0.1 (correctly rejects)

Report example (minimal)


```
{
  "schema_version": "1.0.0",
  "suite_version": "alpha_elasticity_ci_v1",
  "gate_id": "AE-CI-0003",
  "status": "CI",
  "decision_rule_ref": "DECISION_RULE.json",
  "decision_rule_sha256": "...",
  "provenance": {
    "seed": 12345,
    "commit": "abc1234",
    "timestamp_utc": "2025-12-28T12:00:00Z",
    "inputs_sha256": {
      "synthetic_set.tar.gz": "...",
      "meta.json": "..."
    }
  },
  "tests": {
    "T1_recovery_structured": {
      "metrics": { "median_rel_error": 0.031, "cv": 0.021, "n": 40
},
      "pass": true
    },
    "T4_null_adversarial": {
      "metrics": { "F1": 0.91, "FP_rate": 0.03 },
      "pass": true
    }
  },
  "GLOBAL_PASS": true
}
```

Failure modes (explicit)

The suite must fail (or mark invalid) under these conditions:

- Any gate is changed without updating gate_id.
- Missing provenance (seed, commit, inputs_sha256).
- Adaptive gates based on closeness to target.
- Target leakage (features derived from reference alpha or target labels).
- Synthetic generator injects alpha directly into estimator outputs.
- Non-determinism with fixed seed.

What this suite does not claim

- It does not validate SFT physics end-to-end.
- It does not prove alpha is physically derived from SFT.
- It does not replace REAL solver campaigns (P) and convergence/GCI.

It only validates that the alpha-elasticity pipeline is reproducible, falsifiable, and resistant to trivial self-confirmation.

Companion Toy Example (CI Visualization) — How to Use This Suite in Practice

Status and scope (read this first)

This companion example is a **toy visualization module** meant to demonstrate how the **SFT Alpha-Elasticity Validation Suite** works operationally (inputs → metrics → gates → PASS/FAIL → hashed artifacts).
It is **not** a physical derivation of the fine-structure constant and must **not** be used as evidence of physical validation. Its purpose is to validate **auditability**, **anti-leakage discipline**, and **failure-mode coverage** under controlled synthetic inputs.

Core idea of the toy example

The toy pipeline produces a dimensionless **alpha_like** scalar from synthetic field artifacts using fixed, preregistered operators and gates. A recommended default is:

- **alpha_like** := `median( ||H(S)||_F ) / ( median(|∇S|) + eps )`

Note: alpha_like is a demonstration metric for CI validation; it is not asserted to equal the physical fine-structure constant α.

All operators (∇, H) are computed using the preregistered discrete stencil and boundary handling specified in DECISION_RULE.json.

where:

- ∇S is the discrete gradient of the scalar field `S`,
- `H(S)` is the discrete Hessian,
- $||\cdot||_F$ is Frobenius norm,
- `eps` is a fixed numerical stabilizer declared in the decision rule.

This definition is chosen because it is:

- fully internal (no use of 1/137 or any α target),
- dimensionless (ratio of invariants),
- sensitive to controlled changes in synthetic stiffness/roughness,
- and easy to visualize.

Required artifacts produced by the toy runner

This companion demo produces the same core artifacts defined in the suite Outputs section (report, preregistered decision rule, and hash manifest). In addition, it may emit the following non-authoritative plots for visualization:

- alpha_hist.png
- sensitivity_sweep.png
- confusion_matrix.png
- robustness_vs_resolution.png

How this toy example maps to the suite tests (T1–T5)

T1 — Recovery on structured synthetic fields (non-tautological)

- Generate structured synthetic `S` fields from a parameterized generator `G(params, seed)`.
- Compute `alpha_like` for N samples.
- Score against a **locked reference mapping** only for evaluation (not accessible to the estimator).
- Expected output: stable alpha_like distribution with bounded median error and low CV.

T2 — Cross-configuration consistency (repeatability)

- Create K families of synthetic configurations that share the same underlying condition (same generator regime) but vary layout/seed/perturbations.
- Verify that between-family drift remains within gates.

T3 — Sensitivity to controlled parameter change (monotonic response)

- Sweep a single generator parameter `p` (e.g., synthetic stiffness/roughness knob).
- Check monotonicity via Spearman ρ and slope significance (bootstrap CI).

T4 — Discriminative power vs null/adversarial (no adaptive thresholds)

- Build labeled sets:
  - Positive: structured fields.
  - Null: random or phase-scrambled fields.
  - Adversarial: near-structured fields with injected violations.
- **Critical constraint:** Null/adversarial generation must not access `alpha_reference` or target labels; it must operate only on unlabeled synthetic artifacts.
- Evaluate with confusion matrix + F1 + FP_rate under a **fixed** decision rule from `DECISION_RULE.json`.

T5 — Robustness to grid / BC perturbations

- Re-run the same scenario across preregistered resolution levels and mild BC variants.
- Verify stability (or correct rejection) according to the preregistered robustness policy.

PASS/FAIL semantics

- A run is **PASS** only if:
  1. all required provenance fields are present,
  2. `decision_rule_sha256` matches the provided `DECISION_RULE.json`,
  3. input and output hashes are recorded,
  4. all test gates pass, and
  5. `GLOBAL_PASS = true`.

Any missing provenance, missing rule hash, or post-hoc gate change is an automatic **FAIL**.

Why this "toy visualization" matters

- Third-party runnable demonstration of the suite's PASS/FAIL workflow.
- Auditable integrity: preregistered gates (gate_id) and hash-tracked artifacts.

Relationship to other SFT packs

- alpha_like (this CI suite) is a synthetic stability/consistency metric; it is not asserted to equal the physical fine-structure constant.
- alpha_out (A-OUT / Doc 10) is the physical-constant composition pipeline and should be treated as a separate, REAL-ready route.
- Capstone exploratory demos may reference α only for post-hoc comparison; thresholds and decision rules must remain preregistered (no target-driven tuning).

Conflict of Interest

The author declares no conflict of interest.

Funding

This research received no external funding.

Author Contributions

The author was solely responsible for conceptualization, methodology, software, formal analysis, investigation, writing (original draft and review & editing), and project administration. The author was responsible for conceptualization, writing, and preparation of materials.

Data Availability Statement

The data and verification artifacts supporting this study are available in the project repository (see "Repository" section) and in the associated release assets/manifests.

Repository structure

Doc_0/ ... Doc_14/ The full documentation set (each doc in its own folder). CLEANED.zip and runner bundles When a document has an executable verification component, the corresponding zipped runner bundle is included.

Repository Github: https://github.com/Xaquer69/sft-theory-and-runners MIT License

Zenodo DOI: https://doi.org/10.5281/zenodo.18648618